CONTINUOUS VULNERABILITY MANAGEMENT & AUTONOMOUS RED-TEAMING

Vanguard

Vanguard finds weaknesses in your own stack before attackers do — and proves them. For NVIDIA DGX / GB10-class systems and the agentic services that run on them.

THE PROBLEM

Known vulns are a grind. Unknown ones are invisible.

Matching CISA KEV, OSV, NVD and vendor feeds against your actual running stack — with correct Debian version ordering, runtime reachability and OS scoping — is slow, manual and error-prone. And nobody is adversarially probing your own agentic services the way an attacker will: patiently, deviously, until something gives.

LOOP 01 · REACTIVE

"Which known vulns are in my stack right now?"

Ingest KEV / OSV / NVD / vendor → match against installed packages → triage → propose verified fixes.

LOOP 02 · PROACTIVE

"What unknown weaknesses can an attacker find — and prove?"

A frontier-model agent attacks a faithful replica of your own service until it finds a weakness with a machine-checkable proof.

WHY IT'S DIFFERENT

An attacker, not a scanner.

01

A frontier-model attacker

The red-team agent acts, observes and decides the next step — told to be devious and unrelenting, to enumerate and chain findings the way a real adversary behaves. It is not replaying a wordlist.

02

Deterministic proof, not model confidence

The harness — not the model — detects compromise and proves it. Every finding re-executes its repro before it's accepted; unverified claims are rejected.

03

Semantic proof for agentic attacks

Prompt-injection and objective-injection findings are machine-verified by checking the response content — did the agent escalate? leak its instructions? — not just an HTTP status code.

04

Hardened by design

The attacker's toolset is bounded — HTTP to the target only, no shell, no filesystem, hard step caps — so it can never escape to anything it isn't authorised to touch.

IN THE WILD

It found real issues in our own stack.

Pointing Vanguard at a faithful replica of Pentatonic's own control plane, the red-team agent machine-proved an **agentic prompt-injection escalation** — carefully-phrased input that slips past the intent guard and becomes the

agent's objective — and surfaced a **reproducible worker crash** in the authentication path. Both filed and being fixed.

> Two real, non-obvious weaknesses a conventional scanner would never have seen.

GET STARTED

Clone, scan, attack.

```
# reactive CVM against the machine you're on
go run ./cmd/vanguard scan

# hardware inventory — driver, CUDA, cuDNN, GPU firmware, NGC, Grace/GB10, DPU
go run ./cmd/vanguard inventory

# proactive red-team against the in-repo demo target
make dogfood

# local read-only dashboard
make ui
```

Go 1.22+ · Python 3.10+ · Apache-2.0 · github.com/Pentatonic-Labs/vanguard